

k-Nearest Neighbors

Classification without training

Supervised Learning · Classification



five neighbors → blue wins 3–2

The Core Idea

$N_k(x)$ = the k closest training points to x

$\hat{y}(x) = \text{majority}\{y_i : x_i \in N_k(x)\}$

$d(x, x_i) = \|x - x_i\|_2$

```
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
```

```
X, y = make_classification(
    n_samples=200, n_features=2,
    n_redundant=0, random_state=0)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X, y) # stores X and y
print(model.predict_proba(X[:3]))
```

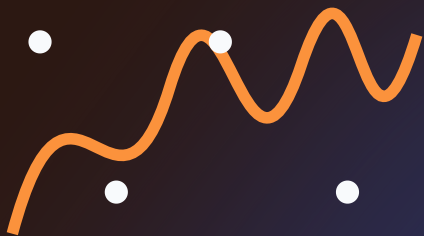
- Find neighbors using a distance metric
- Vote on their labels
- Fit is nearly free; prediction computes the distances
- The probability is the fraction of neighbors in each class

Training
store examples

Prediction
search + vote

Choosing k Is Choosing Complexity

$k = 1$



Low bias · high variance

$k = 15$



Useful middle ground

$k = n$



Majority class everywhere

Choose k by cross-validation. Small k is flexible; large k is smooth.

A Worked Vote by Hand

Query point `q = (2, 3)`, `k = 3`

point	label	$d_2(q, x_i)$
(1, 1)	orange	$\sqrt{5} \approx 2.24$
(3, 5)	blue	$\sqrt{5} \approx 2.24$
(6, 2)	orange	$\sqrt{17} \approx 4.12$
(2, 6)	blue	3.00

- Sort by distance: (1, 1), (3, 5), (2, 6), (6, 2)
- 3 nearest neighbors: orange, blue, blue
- Majority vote: **blue** (2 of 3)

```
import numpy as np

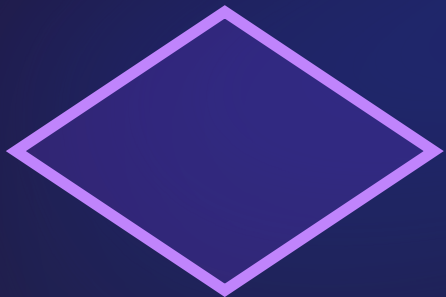
X = np.array([[1, 1], [3, 5], [6, 2], [2, 6]])
y = np.array(['orange', 'blue', 'orange', 'blue'])
q = np.array([2, 3])

dist = np.linalg.norm(X - q, axis=1)
order = np.argsort(dist)[:3]
neighbors = y[order]
vote = np.unique(neighbors, return_counts=True)
print(dist.round(2), neighbors, vote)
```

Distance Defines “Nearest”

$$d_p(x, x_i) = \left(\sum_j |x_j - x_{ij}|^p \right)^{1/p}$$

L1 · Manhattan



Robust to coordinate-wise extremes

L2 · Euclidean



Most common default

L ∞ · Chebyshev



Largest coordinate difference

Distance Metrics in scikit-learn

```
euclidean = KNeighborsClassifier(  
    n_neighbors=5, metric='minkowski', p=2  
)  
.fit(X, y)  
  
manhattan = KNeighborsClassifier(  
    n_neighbors=5, metric='minkowski', p=1  
)  
.fit(X, y)  
  
print(euclidean.predict(X[:5]))  
print(manhattan.predict(X[:5]))
```

p = 2

Euclidean neighborhoods

p = 1

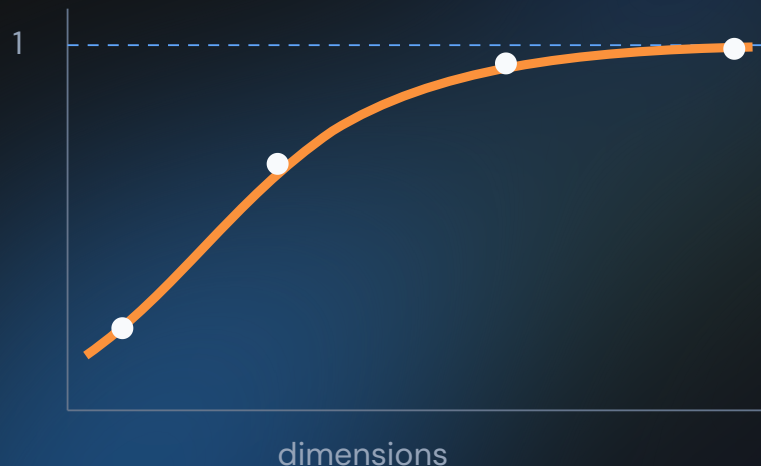
Manhattan neighborhoods

Categorical or mixed features need Hamming or another domain-appropriate similarity.

The Curse of Dimensionality

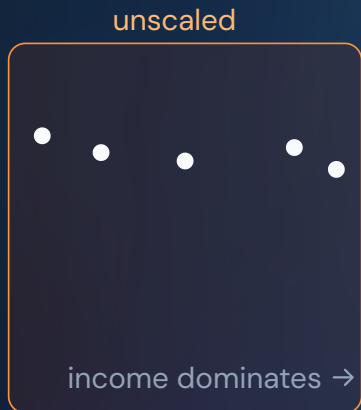
- k-NN assumes nearby points have similar labels
- High-dimensional points spread out and become roughly equidistant
- “Nearest” loses meaning as D grows
- Feature selection or dimensionality reduction can restore useful neighborhoods

$$\frac{r_{\text{nearest}}}{r_{\text{farthest}}} \longrightarrow 1 \quad \text{as } D \rightarrow \infty$$



```
rng = np.random.default_rng(0)
for d in [2, 10, 100, 1000]:
    points = rng.uniform(0, 1, (1000, d))
    query = rng.uniform(0, 1, d)
    dist = np.linalg.norm(points-query, axis=1)
    print(d, dist.min()/dist.max())
```

Feature Scaling Is Essential



- Distance is dominated by the largest numeric scale
- k-NN has no learned weight that can compensate
- Standardize inside a pipeline so each CV fold fits its own scaler

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X, y)
```

Cheap Fit, Expensive Prediction

	fit	predict
k-NN	store	$O(nd)$
logistic	optimize	dot product

- Keep the whole training set in memory
- KD-trees and ball trees help in low/moderate dimension
- k-NN regression averages neighbor targets instead of voting

```
from sklearn.neighbors import KNeighborsRegressor
import time

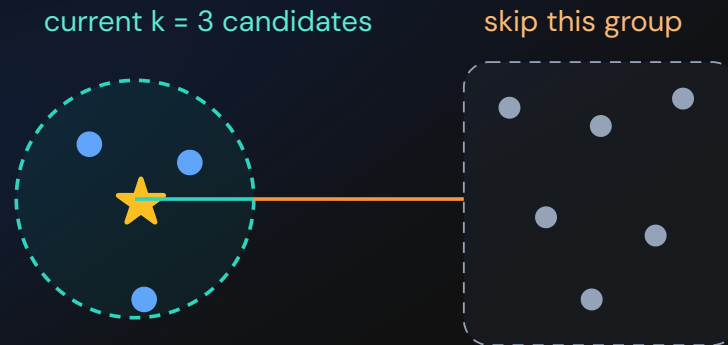
model = KNeighborsClassifier(
    n_neighbors=5, algorithm='kd_tree'
).fit(X, y)
start = time.perf_counter()
model.predict(X)
print(time.perf_counter() - start)

regressor = KNeighborsRegressor(n_neighbors=5)
```

Why Organize the Search?

- Brute force measures distance to every point
- A search tree organizes points into smaller groups
- Search nearby groups first; keep the best k candidates
- Skip a group only when it cannot contain a closer point

Build the index once during `fit` ; reuse it for many predictions. The neighbor vote stays the same.



Even its nearest edge is too far away.

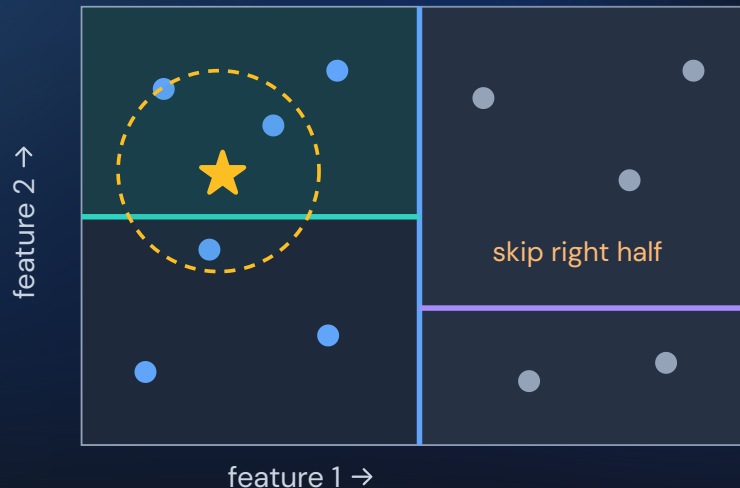
KD-Trees: Split Space into Boxes

- Choose a feature and split points at a cutoff
- Repeat within each group to form smaller boxes
- Search the query's box, then other plausible boxes
- Skip boxes farther away than the current k candidates

KD means k-dimensional: here that refers to the number of features, separate from k-NN's neighbor count.

Often useful with a modest number of numeric features.

```
algorithm='kd_tree'
```



Circle crosses split: also check the lower box.

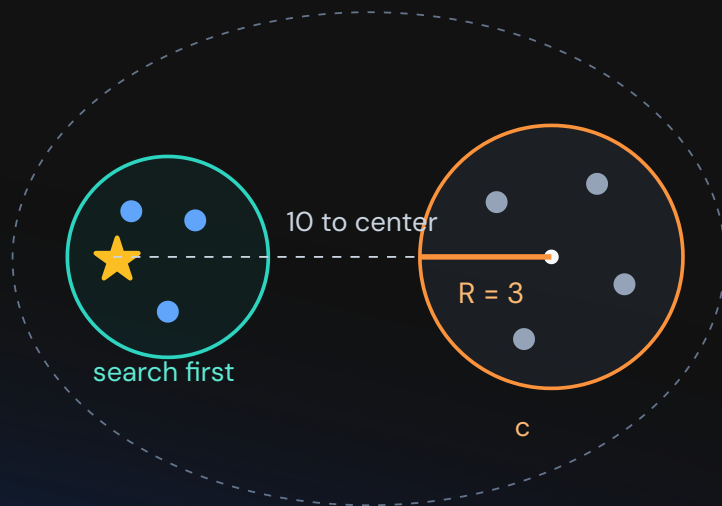
Ball Trees: Bound Groups with Spheres

- Store each group inside a ball: center c , radius R
- Divide its points into smaller groups; repeat
- Search promising balls first
- Skip a ball if even its closest possible point is too far

$$\text{minimum possible distance} = \max(0, \|q - c\| - R)$$

Can suit clustered geometry; speed depends on the data.

`algorithm='ball_tree'`



$10 - 3 = 7$ away at minimum: skip if current k neighbors are all within 2 units of the query.

k-NN in One View

Rule

vote among the k nearest stored examples

Complexity

small k flexes; large k smooths

Geometry

metric and scaling define “near”

Caveat

high dimensions and large n make prediction hard

Next: **Naive Bayes** — a generative classifier built directly from Bayes’ theorem.